GraphRAG Explorer

Wem gehört was im österreichischen Bankwesen, und wie findet man das heraus?

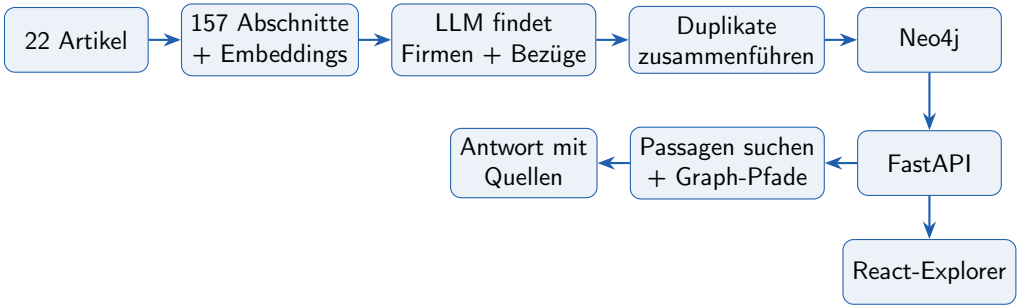
Kacper Zaleski

Was ich bauen wollte

- Eine App, in der man sich **durchklicken** kann, wie Banken, Eigentümer und Aufsicht zusammenhängen, und dabei auf Dinge stößt, die man nicht wusste
- Zu jedem Fakt gibt es **den Satz, aus dem er stammt**
- Und eine ehrliche Frage: Liefert der Graph wirklich **bessere Antworten** als normales RAG?

Daten	22 Wikipedia-Artikel über österreichische Banken
Stack	Python, FastAPI, Neo4j, React + Cytoscape.js, Docker
LLMs	Gemini (kostenlos); Groq, Cerebras oder Mistral mit einer Einstellung

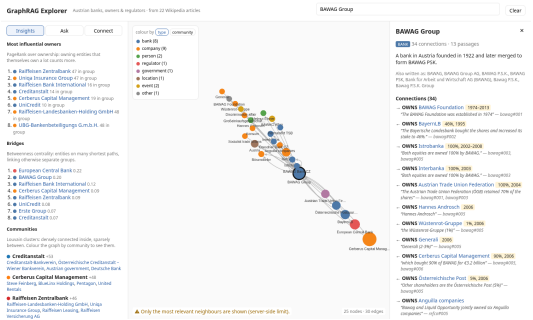
So hängt alles zusammen



- Ergebnis: **726 Einträge und 585 Verbindungen**, jede mit Verweis auf ihre Textquelle
- Ein zweiter Import kostet nichts: jeder LLM-Aufruf wird zwischengespeichert

Ich zeig's Ihnen

1. Wo anfangen: die größten Eigentümer und Gruppen
2. BAWAG: wem sie gehörte, wann und zu wie viel
3. Wie hängt der ÖGB mit GoldenTree zusammen?
4. Wie hängt Refco mit Cerberus zusammen?
5. Frage: *Who have been the owners of BAWAG over time?*
6. Frage: *How is BayernLB connected to Addiko Bank?*



Vom Text zum Graphen

- Das LLM liest jeden Abschnitt und füllt ein **festes Formular** aus: wer kommt vor, wie hängen sie zusammen
- Nur 8 Arten von Einträgen und 9 Arten von Verbindungen, damit alles vergleichbar bleibt
- „A ist eine Tochter von B“ und „B besitzt A“ werden **dieselbe Verbindung**
- Das LLM muss **seinen Beleg zitieren**, und der Code prüft, ob das Zitat wirklich im Text steht
- Eine falsche Verbindung fliegt raus, nicht gleich der ganze Abschnitt

Was schiefging: Meine erste Prüfung wollte das Zitat wortwörtlich und hat 73 Verbindungen verworfen. Rund 70 davon waren richtig, das Modell hatte das Zitat nur mit „. . .“ gekürzt. Wort für Wort prüfen hat's gelöst, und verschluckte Umlaute fallen trotzdem auf („Österreichische“ → „sterr ischische“).

Ist „ÖGB“ dasselbe wie „Austrian Trade Union Federation“?

- Zuerst einfache Regeln: Groß-/Kleinschreibung, Akzente, ö→oe, „AG“, „GmbH“ usw.
- Der Rest kommt in eine **kurze Alias-Liste, die ein Mensch geprüft hat**
- Im Zweifel **lieber getrennt lassen**: zwei Raiffeisenbanken fälschlich zusammenzulegen würde einen Eigentümer erfinden, den es nicht gibt

Was schiefging: Die wichtigste Kette, ÖGB → BAWAG ← Cerberus, war unterbrochen, weil das LLM mal die Kurz- und mal die Langnamen geschrieben hat. Kleines Detail am Rande: Wikipedia selbst schreibt einmal „Ceberus“.

Eine Frage mit dem Graphen beantworten

1. Den Text auf zwei Arten durchsuchen, nach Bedeutung und nach Stichworten, und **die Ranglisten kombinieren**
2. Erkennen, welche Banken und Firmen in der Frage vorkommen
3. Den **kürzesten Weg** zwischen ihnen finden, dann ihre Nachbarschaft anschauen
4. Orte überspringen: „beide sitzen in Wien“ ist keine brauchbare Verbindung
5. Die besten Passagen auswählen und **immer die behalten**, die den Weg belegen
6. Mit Quellen antworten. Zitiert das Modell etwas, das es nie gesehen hat, entfernt der Code es

Was schiefging: Zuerst habe ich stark vernetzte Knoten gesperrt, damit Wege nicht über Knotenpunkte laufen. Aber hier *sind* die gut vernetzten Banken die Antwort. Funktioniert hat, Orte zu überspringen.

Was das Entdecken spannend macht

- **Wann und wie viel**: ein zweiter LLM-Durchlauf ergänzt Jahre und Anteile bei rund 200 Eigentumsverbindungen
  BayernLB 46 % (1995) → ÖGB 100 % (2004) → Cerberus 90 % (2006) → GoldenTree 21,8 %
- **Wo anfangen**, wenn man nichts weiß:
  - Wem gehört am meisten (PageRank über die Eigentumsverhältnisse)
  - Wer verbindet sonst getrennte Gruppen (EZB, BAWAG)
  - Welche Banken zusammengehören (Erste mit ihren CEE-Banken, UniCredit mit Bank Austria)
- Wege finden braucht kein LLM, geht also sofort und kostet nichts

Was schiefging: Ein Kartenzahlungs-Anbieter namens SIA tauchte unter den fünf größten Eigentümern auf. Aus „UniCredit verkaufte ihr Kartengeschäft an SIA“ war „SIA kaufte UniCredit“ geworden. Eine einzige falsche Verbindung hat das ganze Ranking verzerrt.

Leben mit kostenlosen APIs

- Die APIs waren oft langsam, überlastet oder das Kontingent war aufgebraucht
- Deshalb: Timeouts, Wiederholungen, ein eigener **Rate Limiter** und ein **Ersatzmodell**, wenn eins ausfällt
- Jede Antwort wird **zwischengespeichert**: Wiederholungen sind gratis und liefern dasselbe
- Bricht ein Import mittendrin ab, wird nichts geschrieben, und der nächste Lauf macht dort weiter

Was schiefging: Das Modell, das ich wollte, erlaubt **20 Anfragen pro Tag**. Der Import braucht 160. Und bei Embeddings zählt jeder Text in einem Batch als eigene Anfrage.

Wie ich es getestet habe

- 28 Fragen: 13 mit mehreren Schritten, 10 einfache Nachschlagefragen, 5 über alle Artikel hinweg
- Fragen aus den **Artikeln geschrieben, nicht aus dem Graphen**, sonst bewertet sich der Graph selbst
- Ein LLM bewertet die Antworten, aber **von einem anderen Anbieter** als das antwortende, weil Modelle ihren eigenen Stil gern mögen
- Ein zweiter Bewerter kontrolliert den ersten
- Beide Modi beantworten **dieselben Fragen**, ich vergleiche also Frage für Frage

Runde 1: unentschieden

	normales RAG	Graph
Score [95 %-KI]	0,91 [0,80; 0,98]	0,91 [0,84; 0,98]
richtig	24/28	23/28
richtige Passagen gefunden (Recall@8)	0,90	0,89
Antwortzeit / Tokens	1,7 s / 2,8k	2,5 s / 3,4k

- Überhaupt kein Unterschied (+0,00, McNemar $p = 1{,}0$)
- Die Fragen waren **zu leicht**: sie nennen alles beim Namen, also findet es auch die normale Suche
- Beide scheitern an „Nenne alle . . .“-Fragen: in 8 Passagen passt keine vollständige Liste

Runde 2: schwerere Fragen, Vorhersage vorher aufgeschrieben

- Meine Wette, **vor dem Lauf aufgeschrieben**: der Graph gewinnt, wenn das Zwischenglied nie genannt wird
- 8 Fragen wie „How is A connected to C?“
- Nach den Ergebnissen habe ich nichts mehr verändert

	normales RAG	Graph
Score	0,38	0,44
richtig	3/8	2/8
richtige Passagen gefunden	0,56	0,50
… **ohne Reranker**	**0,81**	0,56

- +0,06, aber irgendwo zwischen −0,12 und +0,25: **meine Wette ging nicht auf**
- Der größte Effekt war eine Überraschung: der **Reranker wirft genau die Passagen weg**, die normales RAG braucht
- Die ursprünglichen 28 funktionieren weiter (Graph 0,93, normales RAG 0,91), und die beiden Bewerter sind sich bei 64 von 72 Antworten einig ($\kappa = 0{,}62$ / 0,69)

Was ich mitnehme

Wo sich der Graph lohnt

- Erkunden: Wege, Nachbarn, Rankings
- Vertrauen: jede Verbindung zeigt, woher sie kommt
- Zeit und Anteile bei Eigentum
- Fehler sieht man sofort (SIA, Cerberus)

Wo (noch) nicht

- Bessere Antworten bei wenigen, sauberen Artikeln
- Er weiß nur, was extrahiert wurde: „Addiko wurde aus der Hypo abgespalten“ ist nie angekommen
- „Nenne alle. . .“ braucht eine Graph-Abfrage, keine Suche

„Kein Unterschied“ ist auch ein Ergebnis, wenn man es sauber gemessen hat.

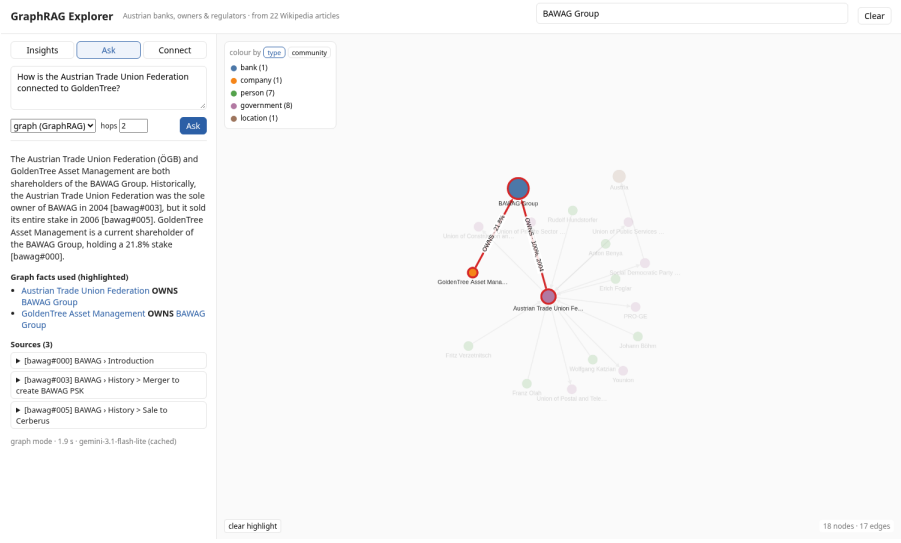
Was ich als Nächstes machen würde

- Den Reranker im Graph-Modus weglassen (er wurde auf Websuchen trainiert, nicht auf solche Fragen)
- „Nenne alle. . . "-Fragen mit einer Graph-Abfrage beantworten statt mit einer Suche
- Abschnitte mit wenigen Verbindungen ein zweites Mal lesen, damit weniger Fakten verloren gehen
- Ein Zeitregler im Explorer (die Jahre sind schon in den Daten)
- In einer Bank: Modelle in der EU oder im eigenen Haus, Snowflake als Quelle, Airflow für regelmäßige Importe

Danke! Fragen?

Code, Evaluation und Notizen liegen im Repository
`github.com/kacperzaleski/graphrag-explorer`

Backup: eine Antwort mit hervorgehobenem Weg



Backup: einen Weg finden

